

Big data Analytics:-

Distributed Computing Systems

→ DFS:- It is a storage system that spreads data across multiple machines in a cluster. It ensures that Data is stored locally on the disks of different nodes.

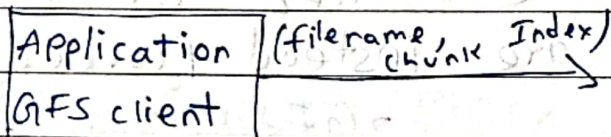
Approach:- Start the worker on the machine where data is already stored, it is faster & reduces network load.

Need:-

- Network bandwidth is limited.
- RAM is insufficient for huge datasets.

ex: GFS, HDFS

GFS (Google File Systems) Diagram:-



GFS: * Assumptions:-

- Uses commodity-hardware (cheap, fail-prone)
- Scales out, not up (Add machines not upgrade)
- Handle modest number of very large files.
- Optimized for large streaming reads.

Properties

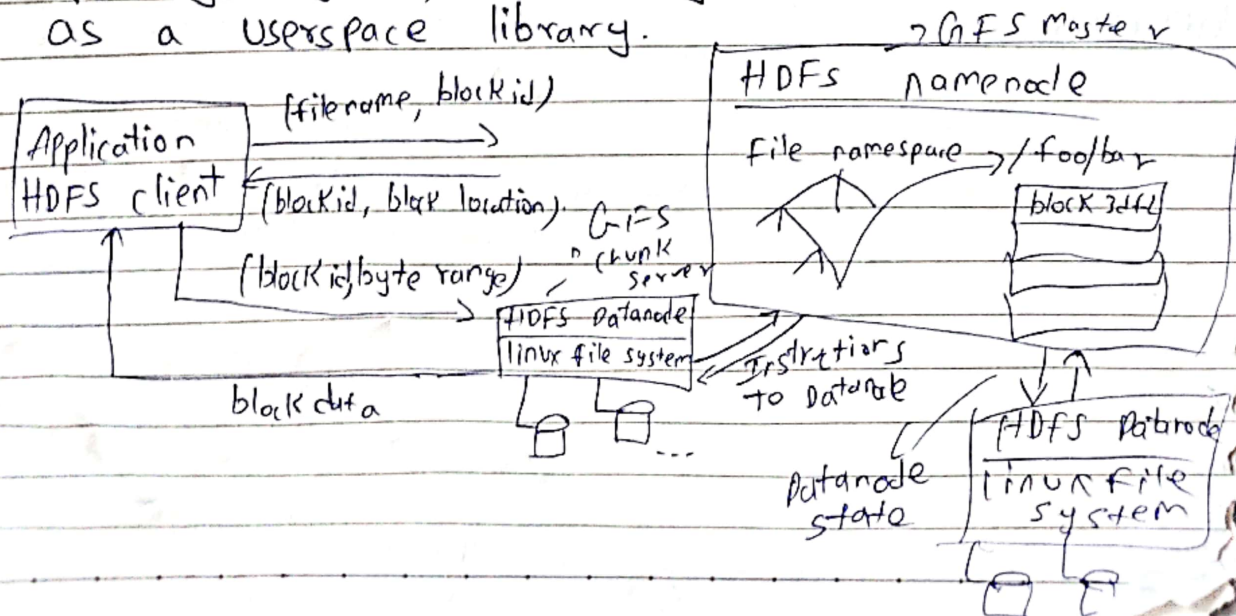
* Design Decisions:-

- Files split into fixed-size chunks (64mb)
- Replication for fault tolerance.
- Single master contains metadata & coordinate access.
- Chunk servers store Actual data.
- Simple API / No data caching.

HDFS → Java Implemented
GFS Inspired

GFS TO HDFS:-

→ They are Implemented in the Kernel of an operating system, but they are Instead Provided as a userspace library.



Resource sharing in Hadoop.

• Coarse-Grained Sharing :-

- Allocate an entire machine to a framework for its whole lifetime
- Used in Hadoop 1 (MapReduce v1)

Problems:

- Poor resource utilization
- Bad Interactivity
- Data locality degrades if machines are held for too long.

- Works only when workloads are uniform & Predictable

• Fine-Grained Sharing :-

- Break work into small tasks and multiplex them across all nodes.
- Multiple frameworks share the same cluster simultaneously.
- Used in Hadoop 2 & 3 (via YARN)

Advantages:

- Better cluster utilization
- Better Interactivity
- Much better for dynamic/mixed workloads.
- Supports multiple frameworks.

1 X

Big data & Hadoop

SV's → Volume, variety, velocity, Veracity, value

Data analytics Tech visualization.

**** Hadoop :-**

- APACHE Hadoop is an open-source framework for reliable, scalable, distributed computing.
- Designed for distributed Processing of large datasets.

Features:

- Data locality
- Scalability
- linear performance Improvement
- Fault tolerance
- High availability via replication.

Components: → HDFS

- MapReduce
- Ecosystem Tools
 - Pig - High level data Processing
 - Hive - SQL Like querying
 - HBase
 - ZooKeeper - coordination & Synchronization.

Hadoop Distributed File System (HDFS)

→ Architecture :-

- master - slave Architecture
- Namenode (master)
- Data node (slave)
- Clients

Working: → Read: - ① Client asks Namenode for the block locations
 ② Namenode returns Datanode address
 ③ Client directly reads data from Datanodes.

→ Write: - ① Client asks Namenode for (metadata + placement)
 ② Name Node chose Datanodes
 ③ Client writes data (Replicated across Data Nodes)
 ④ Namenode records metadata.

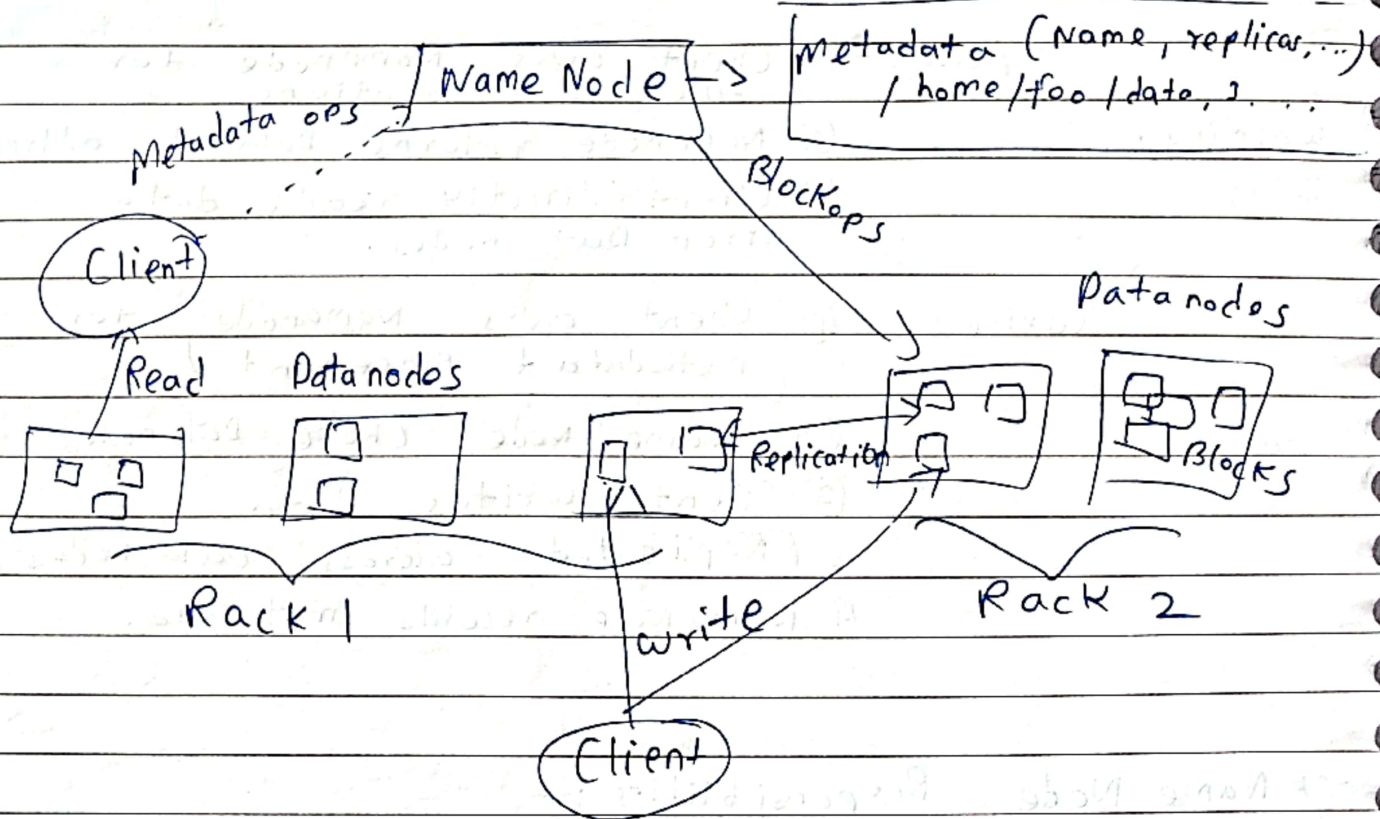
* Name Node Responsibilities :-

- Manages file-System name space
- Maintains metadata File → Block → Data Node
- Controls replication factor
- Monitors Data Nodes, also directs it for Read/write operations.
- Handles Block replication, Load Balancing, Garbage collection.

* Data Node Responsibilities :-

- Stores Actual HDFS Blocks
- Serve read/write requests from clients.
- Perform replication when Instructed.
- Handles failures / rebalancing nodes.
- Ensures Data Integrity.

HDFS Architecture



Map Reduce:-

- It is a Programming model and Implementation for Processing & generating large data sets.
- Designed to process terabytes of data.
- Highly, scalable & fault tolerant.
- Data shuffling, Sorting, Parallel execution across the machines.

→ User defines two functions:-

- Map (Key, value)
 - Produces Intermediate Key-value pairs
- Reduce (Key, values)
 - Merges all values associated with the same Key.

Map-Reduce :-

→ Input-Output Model :-

Data represented as (Key, value) Pairs

Map : $(K_1, v_1) \rightarrow \text{list}(K_2, v_2)$

Reduce : $(K_2, \text{list}(v_2)) \rightarrow \text{list}(K_3, v_3)$

- All values with same Key are sent to the same Reducer.

Input Splitting & Distribution

Start of Individual Map Tasks

Shuffle, Partition/Sort Per Map Output

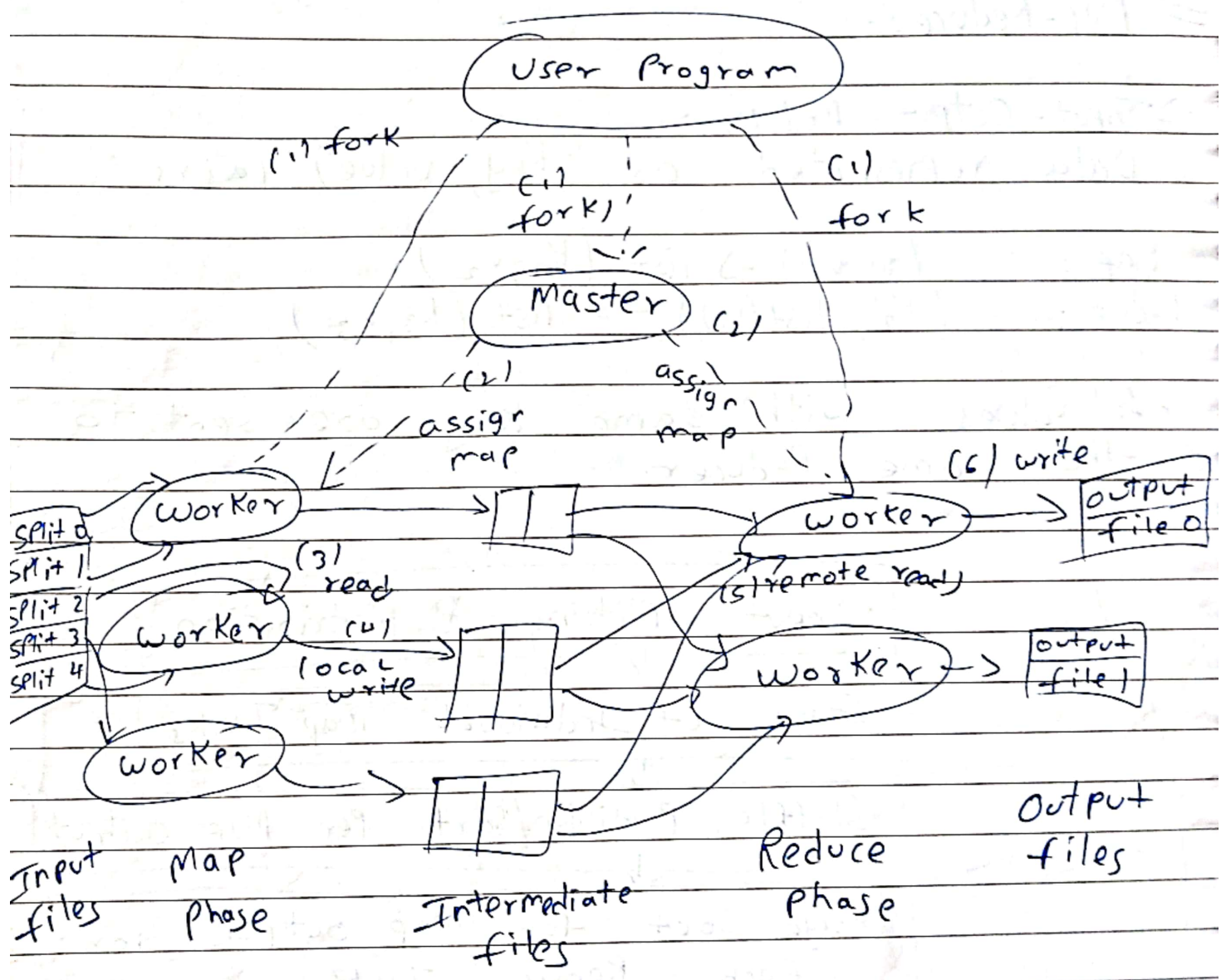
Merge Sort for Map outputs for Each Reduce Task

Start of Individual Reduce tasks

Collection of Final output

- Map-Reduce Runtime Responsibilities :-
- Scheduling : Assign workers to Map & Reduce Tasks
 - Data distribution : Moves computation close to data.
 - Synchronization : Shuffle, Sort & group Intermediate data.
 - Fault Tolerances
 - Runs on Top of HDFS.

Map Reduce Architecture :-



Word count :-

Big data Analytics:-

• Apache storm ->

- > It is a free, open-source distributed real-time computation system.
- > Designed for Processing unbounded streams of data reliably.
- > Capable of Processing millions of Tuples per second per node.

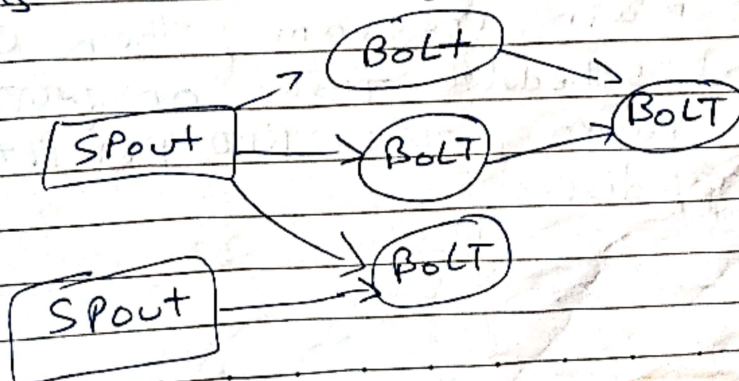
Uses: $\begin{cases} \rightarrow \text{online ml} \\ \rightarrow \text{continuous computation} \\ \rightarrow \text{distributed RPC} \end{cases}$

Storm Topology ->

- > It is equivalent to MapReduce Job but MapReduce finishes whereas storm Topology runs forever until Killed.
- > It is a directed graph of spouts, bolts

Streams

- > It is an unbounded sequence of Tuples.
- > They are created & processed in parallel.
- > Each stream has a schema, uses data types and assigned a unique Stream ID



*# SPoots : -

- Sources of data in a Topology
- Reads data from external systems.

Types: -

- Reliable (Replays Tuples if processing fails)
- UnReliable (Does not track tuple processing)

*# BOLTS : -

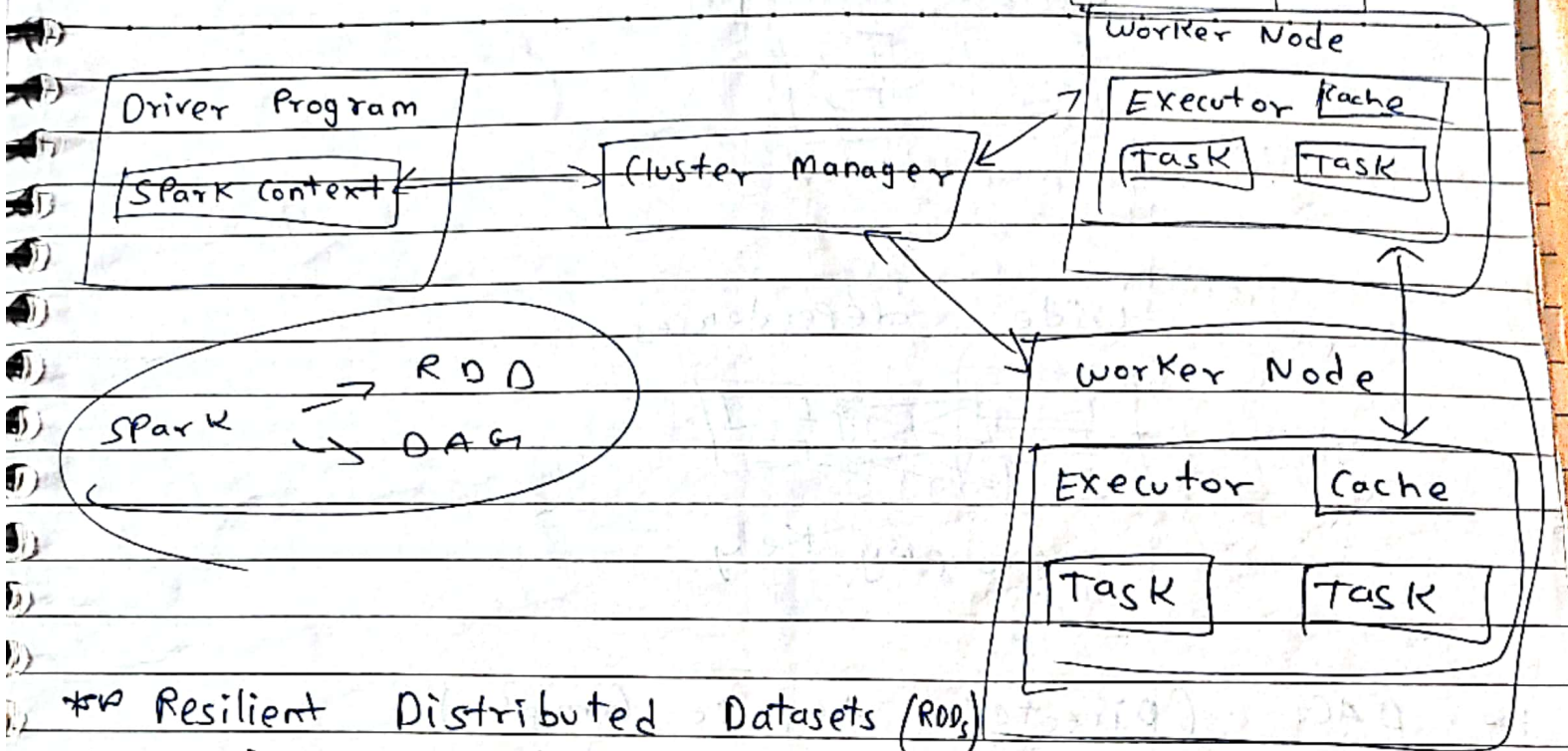
- ALL computation happens in bolts.
- They can filter data, Apply functions, execute Joins, Perform aggregations, write to databases.
- Bolts work reliably & in Parallel.

for complex logic → Multiple bolts in sequence.

ex. tweet → filter → count → Rank → Trending
= Stream Topics.

Apache Spark - Architecture :-

- It follows a Master-slave architecture.
- Master (Driver), Slave (Multiple worker nodes)
- Driver runs the main Program, then Creates the DAG and schedules tasks on workers.
- Workers execute tasks, store RDD partitions in the memory/disk.



* Resilient Distributed Datasets (RDDs)

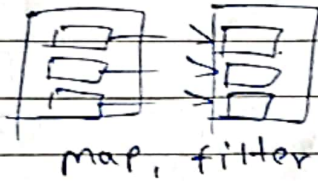
- > It is a Distributed in memory data abstraction.
- > enables in-memory computation across the large clusters.
- > Fault tolerant, Immutable, Partitioned

Each RDD is built from other RDDs using Dependencies:

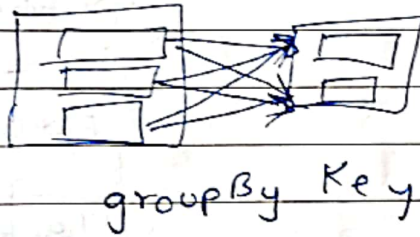
1) Narrow -> Each Parent Partition is used by at most one child Partition
 -> No data shuffle
 ex: map, filter.

2) Wide -> Multiple Child Partitions depend on the same Parent Partition
 -> Requires shuffle.
 ex: reduceByKey, join

Narrow dependencies



wide dependencies



↳ DAG (Directed Acyclic Graph)

→ Built using RDD lineage

→ Each node represents an RDD partition

→ Spark Represents computations as DAG

execution: - ① User calls an action

② Spark Scheduler:-

↳ Analyzes lineage graph

→ Build a DAG of stages

③ Stages

↳ contain only narrow dependencies

④ Stage Boundaries:-

↳ wide dependencies.

⑤ Scheduler recomputes only missing partitions if failures occur.

fault
tolerance

Bolts + execution: --

- All computation happens here. (output) time.
- Takes Input from streams → Processes → outputs new streams.

Bolts functions:

→ Filtering, Transformations, Aggregations, Joins
Database Interaction

Operations: Filter → Pass only valid Tuples
Join → Combine 2 streams
Transform → Apply function to each tuple.

* Bolt Parallelism:

- A Bolt can have multiple tasks (parallel workers)
- Each tuple goes to a task based on grouping

Grouping Types:

1. Shuffle grouping

→ Even distribution

→ Load balancing

2. Fields grouping

→ Group a stream by a subset of its fields.

ex: A-M → Task 1, N-Z → Task 2

3. All grouping

→ every task gets all tuples

→ Useful for joins

4. Global grouping

→ Pick Tasks with lowest id.

Storm API functions :->

- emit (tuple) -> send output
- ack (tuple) -> processing successful
- fail (tuple) -> processing failed

-> Every tuple must be ack/fail -> otherwise memory leak can happen.

Storm @ Twitter :->

-> Apache Storm is a distributed - real Time processing system used to handle unbounded streams of data.

-> At Twitter, Storm is used to process 20 millions of tweets in real time for trend detection, hashtag ranking, and analytics.

Why storm -> handles high-velocity tweet streams

- > Performs real-time analytics
- > ensures fault-tolerance & scalability
- > Supports continuous computation.

Architecture :-

1. Spouts -> Source of data (tweets)
-> Reads data from Queue/API

2. Bolts -> Perform processing
(parsing tweets, filtering, Aggregation, Ranking hashtags)

3. Topology -> Directed graph of spouts & bolts
-> Runs continuously.

Edge Computing, Cloudlet, Fog Computing.

→ Edge Computing is a distributed computing Paradigm where Computation and Storage are Performed close to the user/data source reducing latency & bandwidth usage.

→ Cloudlet is a small scale data center at the edge of the network. It acts as a Cloud near the user and Provides low-latency high-Performance Computing for mobile application.

→ Fog computing, Creates a hierarchical system between Cloud & edge devices. It distributes Computation, storage & networking across the multiple layers.

Edge Caching

→ Content Delivery Network.

→ CDN, It is a geographically distributed network of servers that delivers Content to users from location closer to them.

→ Instead of fetching data from single Central server, CDN redirects user requests to nearby edge users for faster access & less network congestion.

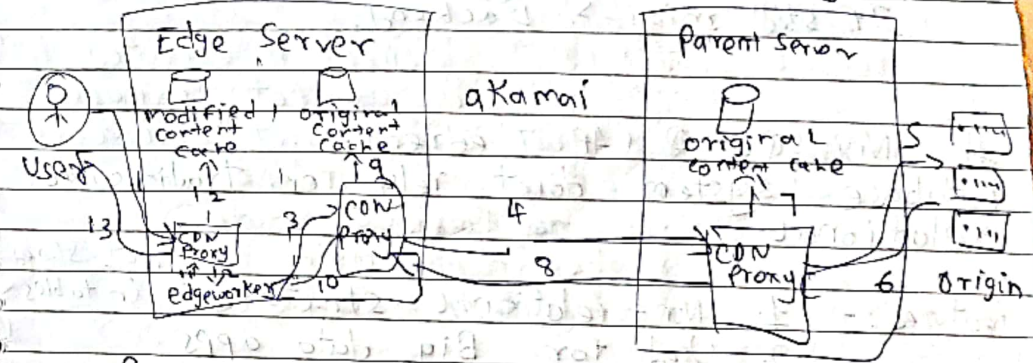
Working :
→ user requests a webpage
→ Request is redirected to nearest CDN server
→ Content is delivered from nearby location
→ Reduces delay & bandwidth usage

* Akamai CDN

→ One of the largest CDN Providers.

→ Uses distributed edge servers worldwide.

→ Facebook uses Akamai's multi-layer caching architecture to serve content efficiently.



1. Browser Cache

→ first level cache inside user service
→ stores frequently accessed data locally.

2. Edge Cache: (CDN/ISP)

→ located close to users
→ handles most requests & reduces latency

3. Origin cache

→ Central Cache layer
→ handles requests not found in edge

4. Backend (Hagstack storage)

→ Stores actual data
→ Accessed only when all cache miss.

flow:

→ Request from user

↓
edge (CPU)

If miss → Origin Cache of distributed
If still miss → Backend.

No SQL & CAP Theorem

Database system don't rely on Traditional relational.

Not Stored
features:-
1. Non-relational structure in tables
2. Used for Big data apps
3. May also support SQL
4. Dynamic schema for the unstructured data.

Properties:-
1. High Scalable (horizontal scaling)
2. CAP Theory.
3. No fully follow ACID Properties
4. No Need of the data handling
5. No Standard Query language.

Types:-
1. Column-oriented Databases
2. Document-oriented Databases
3. Key-Value Databases ex: DynamoDB
4. Graph Databases ex: Redisgraph
5. Multi-model Databases -

Per's betw
data

SQL Transactions:->

→ Single logical unit of work in Database.

ex: \$50 from A → B

1. Read A

2. $A = A - 50$

3. write(A)

4. Read(B)

5. $B = B + 50$

6. write(B)

* Issues:-

1. Failures

(Hardware, Software)

2. Concurrent executions.

(Multiple transaction same time)

ACID:->

A (Atomicity): Transaction fully completes or nothing happens

ex: A debited but B not credited.

Sol: Rollback.

C (consistency): DB must remain valid.

Before T: $A + B = 1000$

After T: $A + B = 1000$

I (Isolation): T should not affect each other.

ex: T1 T2

read(A)

A: $A - 50$

write(A)

read(B)

B: $B + 50$

write(B)

read(A), read(B), Print(A+B)

updated A, but not B

D(Durability) :- once T is completed, changes are permanently stored.

Transaction state :-

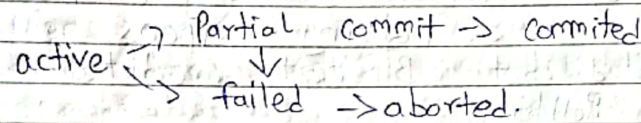
1. Active : init state

2. Partial committed : Statement is executed but changes are not permanent.

3. failed state : Transaction cannot continue.

4. Aborted : Transaction is undone, db restores to previous state.
Kill the Transaction

5. Committed : T successfully completed.



Concurrent execution :->

T1-T2 runs together

Advantages :- (i) Resource utilization.
(ii) less response time.

Disadvantage : -> Two T overlaps.

For Avoiding overlap, To maintain consistency There is concurrency control.

— ^ —

* Schedules : Order in which operations of Multiple Transactions are executed.
-> fail T will abort as last statement.

CAP Theorem

-> It is Impossible for a distributed system to simultaneously guarantee Consistency, Availability and Partition Tolerance, Only two of Three can be achieved at a time.

SQL -> ACID, No SQL -> CAP

Consistency : All nodes see the same data at the same time.

ex: MongoDB

Availability : Every request gets a response, Even if some nodes are down System still responds.

Partition : System continues working even if network fails betw servers.

1. (Consistency + Partition)

-> System gives correct data, may deny some requests. ex: Bank systems.

2. (Availability + Partition)

-> System always responds but data may be inconsistent.
ex: Social media

3. (Consistency & Availability)

- Works when no partition occurs.
- No practical example bcoz network fails are unavoidable

Consistency

All clients see the same view of data, even right after update/delete

Availability

CA

CP Partitioning

X

All clients can find a replica of data, even in case of partial node failures

AP

The system continues to work as expected even in presence of partial network failure

SQL → ACID

No SQL → BASE (Basically Available, Soft state, eventual consistency)

X